

South Valley National University

Computers and Artificial Intelligence

Level 3 - IT

Micro Controllers

IT301

Course Learning Objectives

By the end of this course, students should be able to:

1. Understand the basic concepts and components of embedded systems.
2. Differentiate between embedded systems and general-purpose computing systems.
3. Describe the architecture of computers and the differences between microprocessors and microcontrollers.
4. Explain the internal architecture and functional blocks of a microcontroller.
5. Analyze the architecture and internal modules of the PIC18F microcontroller.
6. Apply theoretical knowledge to design and implement simple embedded systems using PIC18F.
7. Work with timers, interrupts, and analog-to-digital conversion modules.
8. Configure memory organization, registers, power, and clock sources in microcontrollers.

Learning Outcomes

After studying this content, students will be able to:

Chapter 1: Introduction to Embedded Systems

1. Define what an embedded system is and explain its main characteristics.
2. Compare embedded systems with general computing systems in terms of purpose and performance.
3. Illustrate and describe computer architecture and its main components.
4. Explain microcontroller architecture and identify its internal structure.
5. Distinguish between microcontrollers and microprocessors in terms of design and applications.
6. Classify embedded systems based on their functions and application areas.
7. Identify and describe the main components of an embedded system: processor, memory, I/O ports, peripheral devices, and buses.

Chapter 2: PIC18F Microcontroller Series

1. Describe the PIC18F architecture and identify its key modules.
2. Explain the program and data memory organization in the PIC18F series.
3. Configure and interpret the function of configuration registers.
4. Understand the operation of power supply, reset mechanisms, and the watchdog timer.
5. Operate and program parallel I/O ports and timers.

6. Implement Capture/Compare/PWM (CCP) modules in control applications.
7. Perform analog-to-digital conversion using the A/D Converter module.
8. Design interrupt-driven programs for real-time tasks.
9. Integrate theoretical concepts with practical exercises using the PIC18F microcontroller.

Contents

Contents I
Chabter1:- Introduction to Embedded Systems 1
1.1 What is an embedded system? 1
1.2 Embedded systems vs. General computing systems 1
1.3 Computer Architecture 3
1.4 Microcontrollers architecture 4
1.5 Microcontrollers vs. Microprocessors: 5
1.6 Embedded System Architecture 6
1.7 Embedded System category 6
1.8 Microcontrollers Architecture 8
1.8.1 Microprocessor 8
1.8.2 Memory 9
1.8.3 I/O ports (pin) 10
1.8.4 Peripheral devices 11
1.8.5 Bus 11
Chapter 2: PIC18F Microcontroller Series 13
2-1 PIC18FXX2 Architecture 15
2-2 Program Memory Organization 17
2-3 Data Memory Organization 19
2-4 The Configuration Registers 19
2-5 The Power Supply 24
2-6 The Reset 24
2-7 Power-*on Reset* 26
2-8 The Clock Sources 27
2-9 Watchdog Timer 32
2-10 Parallel I/O Ports 32
2-11 Timers 37
2-12 Capture/Compare/PWM Modules (CCP) 47

2-13 Analog-to-Digital Converter (A/D) Module 55
2-14 Interrupts 62
Exercises 74
Reference 76

Chabter1:- Introduction to Embedded Systems

We interact with embedded devices throughout our day. From devices we use in our morning routines, our modes of transport, to our entertainment systems. In fact, there is no standard definition to distinguish between MCU and embedded. Developers who have developed MCU and embedded have their own definitions.

First clarify the concept, what is a single-chip microcomputer, a single-chip microcomputer is a kind of integrated circuit chip, which uses the ultra-large-scale integrated circuit technology to integrate the central processing unit CPU with data processing capabilities, random access memory RAM, read-only memory ROM, various I/O ports and Interrupt system, timer/counter and other functions (may also include display drive circuit, pulse width modulation circuit, analog multiplexer, A/D converter and other circuits) are integrated on a silicon chip to form a small and complete micro Computer systems are widely used in the field of industrial control. From the 1980s, from the then 4-bit and 8-bit single-chip microcomputers to the current 300M high-speed single-chip microcomputer.

This course gives you an overview of embedded systems, the various steps involved in their design and development and the major domains where they are deployed.

1.1 What is an embedded system?

An embedded system is an electronic/electro-mechanical system designed to perform a specific function and is a combination of both hardware and firmware (software). An embedded system is a computer system with a specific, dedicated function that is not designed so that it should ever need to be reprogrammed (i.e. engine control units, implantable medical devices, appliances, etc.) Every embedded system is unique, and the hardware as well as the firmware is highly specialized to the application domain. Embedded systems are becoming an inevitable part of any product or equipment in all fields including household appliances, telecommunications, medical equipment, industrial control, consumer products, etc. (R1)

1.2 Embedded systems vs. General computing systems

The computing revolution began with general-purpose systems that were designed to handle a wide range of applications. However, it was later realized that embedded systems have different and more specific requirements, such as:

Fast response to stimuli,

Meeting strict computational deadlines,

High power efficiency, and

Operating with limited memory resources.

1. General-Purpose Computing Example

A personal computer (PC) — desktop, laptop, or tablet — is built using a general-purpose processor (e.g., Intel Core or AMD A-Series).

It supports multiple interfaces and peripherals (USB, Wi-Fi, Bluetooth, Ethernet, HDMI, etc.) and allows:

Installing or changing the operating system (OS) (e.g., Windows, Linux, UNIX),

Adding or removing applications (e.g., Microsoft Word, printer software),

Performing multiple tasks simultaneously.

Thus, a general-purpose computer is flexible, programmable, and designed for variety rather than specialization.

2. Embedded System Example

A DVD player, on the other hand, is an embedded system. It is built for a specific task — decoding digital video and producing video output.

Its OS or software cannot be changed or updated by the user.

It lacks general-purpose interfaces like Wi-Fi or Bluetooth.

It contains only the necessary hardware for its specific function (e.g., IR remote control, video output).

Hence, it is application-specific, fixed-function, and not user-programmable.

3. Comparison Table Summary

General-Purpose System	**Embedded System**
Designed for many applications	Designed for specific applications
Uses General-Purpose OS (Windows, Linux, etc.)	May use Embedded OS or none
User can install or remove software	Firmware is pre-programmed and not alterable
Performance (speed) is the main goal	Application-specific needs (power, memory, deadlines) are key
Not optimized for power saving	Optimized for low power consumption

General-Purpose System	Embedded System
Non-time-critical responses	Often time-critical, deterministic responses
Execution can be non-deterministic	Execution is deterministic (especially in real-time systems)

4. Overlap Between the Two

Modern devices like smartphones blur the line between both categories.

They have powerful hardware and multitasking capabilities like PCs,

Yet they remain embedded systems, since their firmware and OS cannot be modified by the end user.

1.3 Computer Architecture

A computer combines a processor, random access memory (RAM), read only memory (ROM), and input/output (I/O) ports. The common bus in Figure 1.26 defines the von Neumann architecture. (R1)

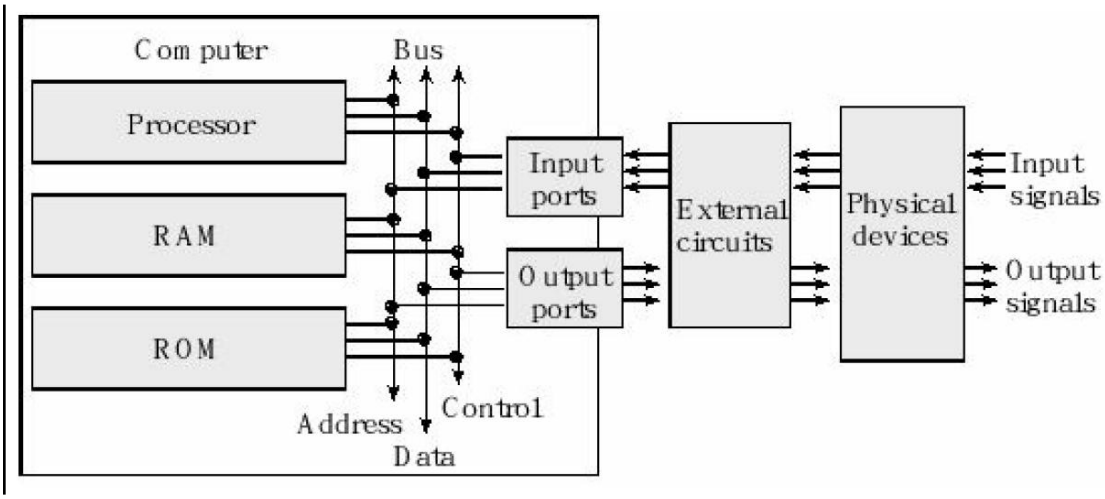


Figure 1.26. The basic components of a von Neumann computer include processor, memory and I/O.

Computers are not intelligent. Rather, you are the true genius. Computers are electronic idiots. They can store a lot of data, but they will only do exactly what we tell them to do. Fortunately, however, they can execute our programs quite quickly, and they don't get bored doing the same tasks over and over again.

Software is an ordered sequence of very specific instructions that are stored in memory, defining exactly what and when certain tasks are to be performed. It is a set of instructions, stored in memory, that are executed in a complicated but well-defined manner.

1.4 Microcontrollers architecture

A microcontroller is a very small microcomputer that integrates all essential components of a computer—processor, memory, and input/output (I/O) ports—onto a single chip. Examples include the Atmel ATtiny, Texas Instruments MSP430, and TM4C123. The term microcomputer can be confusing, as it covers a broad spectrum of systems, ranging from a 6-pin ATtiny4 running at 1 MHz with 512 bytes of memory to powerful 64-bit, multi-core personal computers operating at multi-GHz speeds with terabytes of storage.

A microcontroller can store and retrieve data using RAM (Random Access Memory), which is volatile, meaning its contents are lost when power is removed. Most microcontrollers use Static RAM (SRAM), where each bit is formed by six MOSFET transistors—four configured as cross-coupled inverters to hold the data and two used for read/write operations.

In contrast, ROM (Read-Only Memory) is nonvolatile, retaining information even when power is off. Data retrieval from ROM is similar to RAM, but writing is more complex. Some ROMs are factory-programmed, while Programmable ROMs (PROMs) can be erased and reprogrammed by users, though much more slowly than RAM. Electrically Erasable PROMs (EEPROMs) allow both erasing and programming using electrical signals. Flash ROM, a common form of EEPROM, uses only two MOSFETs per bit—one for charge storage and another for reading—and is typically erased in large blocks. Consequently, most modern microcontrollers use flash ROM to store software (instructions and constants) and SRAM for temporary data and variables.

The TM4C123 microcontroller, for instance, includes 43 I/O pins, while the TM4C1294 has 1024 KiB of flash, 256 KiB of RAM, and 90 I/O pins. External devices connected to these ports expand the system's functionality. An input port allows data to enter the system (e.g., a switch), while an output port sends data out (e.g., controlling an LED). Together, the hardware ports, external circuitry, and software form an interface, enabling communication between the computer and the external environment.

I/O interfaces are generally classified into four categories:

1. **Parallel** – multiple bits transmitted simultaneously across several lines.
2. **Serial** – bits transmitted sequentially on a single line.

3. **Analog** – data represented as continuous electrical quantities such as voltage or current.
4. **Time-based** – data encoded using timing parameters such as period, frequency, pulse width, or phase shift.

Microcontroller represents a complete, compact computing system optimized for control and interaction with the external world, combining processing capability, memory hierarchy, and versatile I/O interfaces on a single chip.

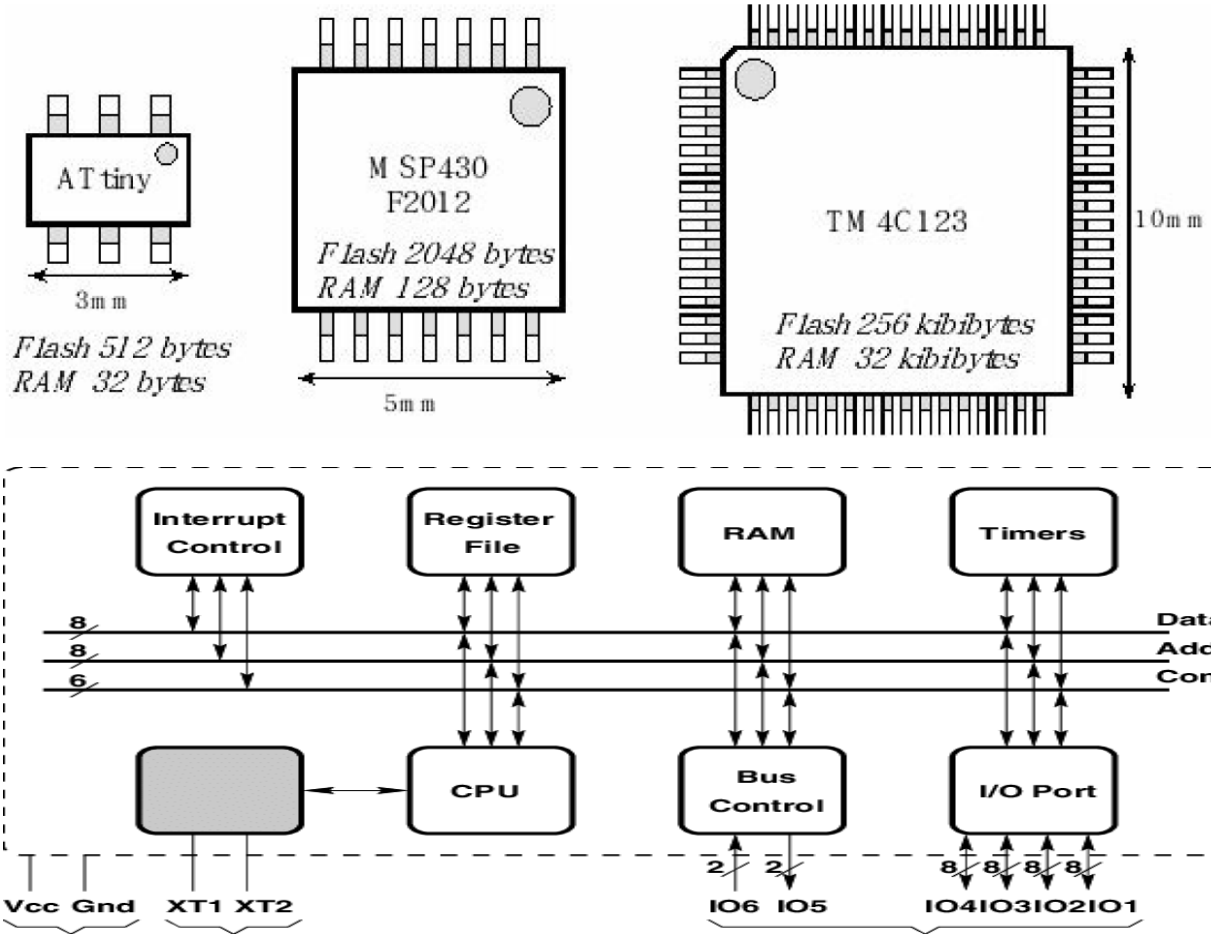


Figure 1.27. A microcontroller is a complete computer on a single chip. The TM4C123 has 43 I/O pins. The TM4C1294 has 1024 kibibytes of Flash ROM, 256 kibibytes of RAM, and 90 I/O pins.

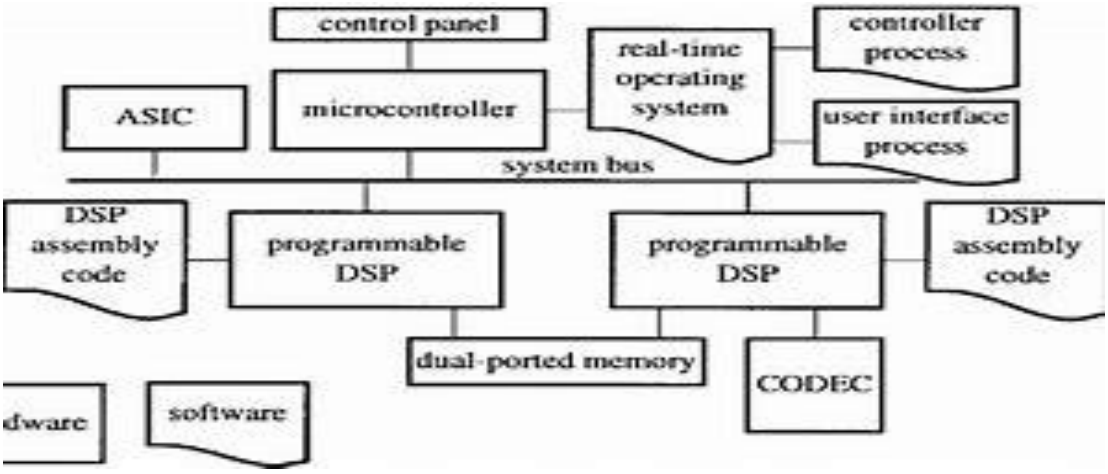
1.5 Microcontrollers vs. Microprocessors:

Microcontrollers are integrated circuits that combine a processor core, memory, and input/output peripherals into a single chip. They are commonly used in embedded systems due to their compactness and cost-effectiveness. (R5)

Microprocessors, on the other hand, are standalone processing units without integrated peripherals. They are typically used in more complex embedded systems where customization and performance are essential.

1.6 Embedded System Architecture

Embedded system architecture refers to the fundamental structure and design of an embedded system, which plays a pivotal role in determining its performance, capabilities, and suitability for specific applications. Understanding the architecture is crucial for developers and engineers working on embedded systems as it influences hardware selection, software design, and system behavior. Here, we will delve into key concepts and considerations related to embedded system architecture. (R5)



1.7 Embedded System category

1- Microcontrollers

The most common type of embedded system is a microcontroller, which is a small computer system on a single integrated circuit. Microcontrollers are adept at performing tasks such as reading sensors and implementing control laws, but it is important to note that these devices are digital, which means they are discretized in how they interpret data, in contrast to the real world in which we live which is analog, so that everything we see is continuous in nature. In order to reconcile this, a microcontroller will utilize both digital-to-analog conversion (DAC) to move from binary values to actual output voltages and analog-to-digital conversion (ADC) to move from an input signal to digital data that the microcontroller can use. (R4)

2- Arduino

Two of the more common microcontrollers are the Arduino. The Arduino Uno is an open-source microcontroller board based on the Microchip ATmega328P microcontroller (MCU) and developed by Arduino.cc and initially released in 2010.[2][3] The microcontroller board is equipped with sets of digital and analog input/output (I/O) pins that may be interfaced to various expansion boards (shields) and other circuits.[1] The board has 14 digital I/O pins (six capable of PWM output), 6 analog I/O pins, and is programmable with the Arduino IDE (Integrated Development Environment), via a type B USB cable.[4] It can be powered by a USB cable or a barrel connector that accepts voltages between 7 and 20 volts, such as a rectangular 9-volt battery. It has the same microcontroller as the Arduino Nano board, and the same headers as the Leonardo board.[5][6] The hardware reference design is distributed under a Creative Commons Attribution Share-Alike 2.5 license and is available on the Arduino website. Layout and production files for some versions of the hardware are also available.

3- FPGA

A field-programmable gate array (FPGA) is a type of integrated circuit that can be programmed or reprogrammed after manufacturing. It consists of an array of programmable logic block and interconnects that can be configured to perform various digital functions. FPGAs are commonly used in applications where flexibility, speed, and parallel processing capabilities are required, such as in telecommunications, automotive, aerospace, and industrial sectors.

FPGA configuration is generally specified using a hardware description language (HDL), similar to that used for an application-specific integrated circuit (ASIC). Circuit diagrams were previously used to specify the configuration, but this is increasingly rare due to the advent of electronic design automation tools.

FPGAs also have a role in embedded system development due to their capability to start system software development simultaneously with hardware, enable system performance simulations at a very early phase of the development, and allow various system trials and design iterations before finalizing the system architecture.[2]

4- Raspberry Pi

The raspberry pi is a small sized computer on a single board that can fit in the palm of your hand. While it might be slower than a laptop or desktop computer, it still has all the capabilities and features that you would expect from a normal computer. **(R3)**

At its size, the raspberry pi definitely punches above its weight!

The *Raspberry Pi* is essentially a computer that can fit in the palm of your hand, without sacrificing any of the functionalities of a typical computer. A raspberry pi is considered an embedded system as it has all requirements of one which includes having a combination of software and hardware such as a processor, memory, and input/output peripheral devices.

Yes, the raspberry pi is an embedded system. This is because the raspberry pi matches the definition of what an embedded system is, which is a system that is a combination of software and hardware used to perform a set of operations within a mechanical or electrical/electronic system.

Due to its number of capabilities and versatility, the raspberry pi has an abundance of uses, and can be found in many different applications.

The raspberry pi is used by the daily DIYer, as well as the experienced engineer. It can be used as a standalone embedded system, or integrated into a bigger system.

Below are some of the many applications where the raspberry pi is used as an embedded system;

- Desktop PC
- Wireless printing network
- Media center
- Gaming servers
- Retro arcade machines
- Robotics
- Image and Video capture
- Music player
- Web servers
- Home automation

1.8 Microcontrollers Architecture

1.8.1 Microprocessor

A microprocessor is basically the brain of the computer. We can also call it simply a processor or CPU. Furthermore, a microprocessor is basically a computer processor that is mounted on a single

IC (Integrated Circuit). It means that all the functions of the processor are included on a single chip. It performs all the processing parts of the computer. It processes the data and instructions which the user gives. Moreover, it carries out the calculations and other such tasks. It has the following parts: **(R6)**

a) Arithmetic and Logical Unit

As the name suggests, this unit is responsible for performing arithmetic tasks like addition, subtraction, multiplication, division moreover, it also makes logical decisions like greater than, less than, etc. And hence the name, the 'brain' of the computer.

b) Control Unit

This unit is responsible for looking after all the processing. It organizes and manages the execution of tasks of the CPU.

c) Registers

These are memory areas which the CPU directly uses for processing. So, it's function is to store data from input or store data between calculations. Besides, it also stores the output results. Moreover, accessing registers is much faster than accessing the RAM.

1.8.2 Memory

A computer contains two types of memory: primary (volatile) and secondary (non-volatile). Primary memory (RAM and ROM) allows quick data access and temporary storage for running programs, while secondary memory (HDDs, SSDs, etc.) provides long-term data storage.

RAM. The term is based on the fact that any storage location can be accessed directly by the processor.

a) Dynamic RAM. DRAM is a type of semiconductor memory that is typically used by the data or program code needed by a computer processor to function.
b) Static RAM. SRAM retains data bits in its memory for as long as power is supplied to it. Unlike DRAM, which stores bits in cells consisting of a capacitor and a transistor, SRAM does not have to be periodically refreshed.

Read-only memory. ROM is a type of computer storage containing nonvolatile, permanent data that, normally, can only be read and not written to. ROM contains the programming that enables a computer to start up or regenerate each time it is turned on.

a) Programmable ROM. PROM is ROM that can be modified once by a user. It enables a user to tailor a microcode program using a special machine called a *PROM programmer*.
b) Erasable PROM. EPROM is programmable read-only memory PROM that can be erased and re-used. Erasure is caused by shining an intense ultraviolet light through a window designed into the memory chip.
c) Electrically erasable PROM. EEPROM is a user-modifiable ROM that can be erased and reprogrammed repeatedly through the application of higher than normal electrical voltage. Unlike EPROM chips, EEPROMs do not need to be removed from the computer to be modified. However, an EEPROM chip must be erased and reprogrammed in its entirety, not selectively.
d) Flash memory

   Flash memory is a type of electronically-erasable programmable read-only memory (EEPROM), but it can also be a standalone memory storage device such as a USB drive.

   It is a non-volatile memory chip used for storage and for transferring data between a PC and other digital devices. It is often found in USB flash drives, MP3 players, digital cameras, and solid-state drives.

   Toshiba developed flash memory in the early 1980s and introduced it to the market in 1984.
e) Flash memory incorporates the use of floating-gate transistors to store data.

1.8.3 I/O ports (pin)

I/O stands for Input/Output.

An I/O port (or I/O pin) is a physical connection point on the microcontroller chip that allows it to communicate with the outside world — such as sensors, LEDs, motors, or other devices.

Input Mode

When a pin is configured as input, the microcontroller reads data or signals from an external device.

Example:

- A button connected to a pin — when pressed, the pin reads a HIGH (1) or LOW (0) signal.

Example:

If a pin is set as input and connected to a temperature sensor, the microcontroller can read the temperature data.

Output Mode

When a pin is configured as output, the microcontroller sends signals to control other devices.

Example: A pin sends a HIGH signal to turn ON an LED, or LOW to turn it OFF.

Port Organization

I/O pins are usually grouped into ports, like: PORTA, PORTB, PORTC, etc.

Each port usually has 8 pins (like an 8-bit register).

1.8.4 Peripheral devices

Peripheral devices are external components connected to a microcontroller (or a computer) to let it interact with the outside world. They are not part of the CPU (core) itself but work with it to perform input, output, or other functions.

A peripheral device is any hardware component that sends data to or receives data from the microcontroller, like LEDs, displays, Sensors, Serial -Counter – ADC – Timer – USB. Without peripherals, the microcontroller would just process data internally — it wouldn't be able to:

- Read sensor data
- Control devices like LEDs or motors
- Communicate with other systems

They make the microcontroller useful in real applications, like robotics, automation, or measurement systems.

1.8.5 Bus

A bus is a set of electrical lines (wires) or paths used to transfer data, addresses, and control signals between the different parts of a microcontroller — such as the CPU, memory, and peripherals.

Main Function

The bus allows different components of the microcontroller to communicate and exchange information efficiently.

Types of Buses

There are three Buses (Date – Addresses – Controls) to send or receive control signals/ data/ addresses

Type	Function	Description
Data Bus	Transfers data	Carries actual information between CPU, memory, and peripherals. Usually bidirectional.
Address Bus	Transfers addresses	Carries the address (location) of where data should be read or written. Usually unidirectional (from CPU to memory/peripherals).

Type	Function	Description
Control Bus	Transfers control signals	Carries control signals (like Read/Write, Enable, Clock) that manage and coordinate all operations.

Example

When the CPU wants to read data from memory:

1. It sends the address on the address bus.
2. It sends a Read signal through the control bus.
3. The data is transferred from memory to the CPU through the data bus.

There are Internal and External Buses. Internal Bus: Connects CPU, memory, and internal peripherals inside the microcontroller. External Bus: Connects the microcontroller to external devices (like external RAM or sensors).